Transactions and Indexes

Module Outcome:

- Wrap an HDFC Bank transfer in a transaction — both rows or neither
- Say ACID in examples, not slogans
- Add an index where a lookup is slow — and not on every column “just in case”

Transactions

A transaction is a logical group of one or more SQL statements. Transactions are used in various scenarios such as banking, ecommerce, social networks, booking tickets, etc.

A transaction has four important properties:

1. Atomicity
2. Consistency
3. Isolation
4. Durability

Atomicity

Either all SQL statements or none are applied to database.

```
✓ Transaction: Debit ✓ → Credit ✓ → ...  (all succeed)
✗ Transaction: Debit ✓ → Credit ✗ → ...  (rollback all)
```

Consistency

Transactions always leave the database in a consistent state.

Example: Transfer of funds between Sam and David.

State	Sam	David	Total
Before	10,000	5,000	15,000
Success	9,000	6,000	15,000
Failure	10,000	5,000	15,000

Isolation

Multiple transactions can occur at the same time without adversely affecting the other.

Example: Concurrent operations on Sam's account:

- Starting balance: **10,000**
- Operations: Debit 1k, Credit 25k, Debit 9k
- Final balance: **25,000** (correct outcome when transactions are isolated)

Durability

Changes of a successful transaction persist even after a system crash.

Example: David's balance is **10,000**. A successful transaction credits **5,000**, bringing the balance to **15,000**. Even if a system failure occurs afterward, the balance remains **15,000**.

These four properties are commonly acronymed **ACID**:

- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability

Transaction Control (TCL)

TCL commands manage transactions — grouping SQL statements into atomic units.

Command	Description
`START TRANSACTION` / `BEGIN`	Starts a transaction
`COMMIT`	Saves all changes permanently
`ROLLBACK`	Undoes all changes since transaction start
`SAVEPOINT name`	Sets a rollback point within a transaction
`ROLLBACK TO SAVEPOINT name`	Rolls back to a specific savepoint

Example: Transfer ₹1,000 from Sam's account to David's account.

Assume an `accounts` table with columns `id`, `name`, and `balance`.

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 1000 WHERE name = 'Sam';
UPDATE accounts SET balance = balance + 1000 WHERE name = 'David';

COMMIT;
```

If any step fails, roll back instead of committing:

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 1000 WHERE name = 'Sam';
-- Suppose this next statement fails (e.g., David's account not found)
UPDATE accounts SET balance = balance + 1000 WHERE name = 'David';

ROLLBACK;
```

After `ROLLBACK`, both accounts remain unchanged — demonstrating **atomicity**.

SAVEPOINT

A **savepoint** creates a checkpoint within a transaction. You can roll back to that point without undoing the entire transaction.

Example: Partial rollback using savepoint.

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE name = 'Sam';
SAVEPOINT after_debit;

UPDATE accounts SET balance = balance + 500 WHERE name = 'David';
-- Suppose we need to undo only the credit, not the debit
ROLLBACK TO SAVEPOINT after_debit;

COMMIT;  -- Sam's debit is kept; David's credit is undone
```

Note: MySQL Workbench and CLI both support `START TRANSACTION`. Some clients auto-commit each statement unless a transaction is explicitly started.

Data Control Language (DCL)

DCL commands manage **user permissions** — who can access which databases, tables, and operations.

Command	Description
`GRANT`	Gives privileges to a user
`REVOKE`	Removes privileges from a user

Common privileges:

Privilege	Allows
`SELECT`	Read data
`INSERT`	Add new rows
`UPDATE`	Modify existing rows
`DELETE`	Remove rows
`CREATE`	Create databases and tables
`DROP`	Drop databases and tables
`ALL PRIVILEGES`	All of the above

Syntax:

```
GRANT privilege_list
ON database_name.table_name
TO 'username'@'host';
```

Example 1: Give a trainee read-only access to the training database.

```
GRANT SELECT ON training_db.* TO 'trainee'@'localhost';
FLUSH PRIVILEGES;
```

Example 2: Give a developer full access to a specific table.

```
GRANT SELECT, INSERT, UPDATE, DELETE ON training_db.product TO 'dev'@'localhost';
FLUSH PRIVILEGES;
```

Example 3: Give all privileges on a database.

```
GRANT ALL PRIVILEGES ON employee_portal.* TO 'admin'@'localhost';
FLUSH PRIVILEGES;
```

REVOKE

`REVOKE` removes previously granted privileges.

Syntax:

```
REVOKE privilege_list
ON database_name.table_name
FROM 'username'@'host';
```

Example: Remove delete permission from a trainee.

```
REVOKE DELETE ON training_db.* FROM 'trainee'@'localhost';
FLUSH PRIVILEGES;
```

> **Note:** `FLUSH PRIVILEGES` reloads the grant tables so changes take effect immediately. Only users with `GRANT OPTION` (typically root) can grant or revoke privileges.

Indexes

In scenarios like searching for a word in a dictionary, we use an index to easily search for the word. Similarly, in databases, we maintain indexes to speed up the search for data in a table.

Dictionary Index Example:

Section	Range	Page
A	ab... – az...	02 – 23
B	ba... – bz...	24 – 32
C	ca... – cz...	33 – 43

Creating an Index

An index is created on one or more columns to speed up `WHERE`, `JOIN`, and `ORDER BY` operations on those columns.

Syntax:

```
CREATE INDEX index_name
ON table_name (column1, column2, ...);
```

Example 1: Create an index on the `brand` column of the `product` table.

```
CREATE INDEX idx_product_brand
ON product (brand);
```

Example 2: Create a unique index to enforce uniqueness (similar to a `UNIQUE` constraint).

```
CREATE UNIQUE INDEX idx_employee_email
ON employee (email);
```

Dropping an Index

Syntax:

```
DROP INDEX index_name;
```

Example:

```
DROP INDEX idx_product_brand ON product;
```

Note: MySQL requires the table name: `DROP INDEX index_name ON table_name;`

When to Use Indexes

Use an index when...	Avoid indexes when...
Columns appear frequently in `WHERE` clauses	The table is very small
Columns are used in `JOIN` conditions	The table has heavy `INSERT` / `UPDATE` / `DELETE` activity
Columns are used in `ORDER BY`	The column has very few distinct values (low selectivity)

Note: Indexes speed up reads but add overhead on writes. Create indexes on columns that are queried often, not on every column.